

Assignment-8.5

HT NO: 2303A51694

Name: P. Niharika

Batch: 24

Task Description #1 (Username Validator – Apply AI in Authentication Context)

- Task: Use AI to generate at least 3 assert test cases for a function `is_valid_username(username)`

and then implement the function using Test-Driven Development principles.

- Requirements:

- o Username length must be between 5 and 15 characters.
- o Must contain only alphabets and digits.
- o Must not start with a digit.
- o No spaces allowed.

Example Assert Test Cases:

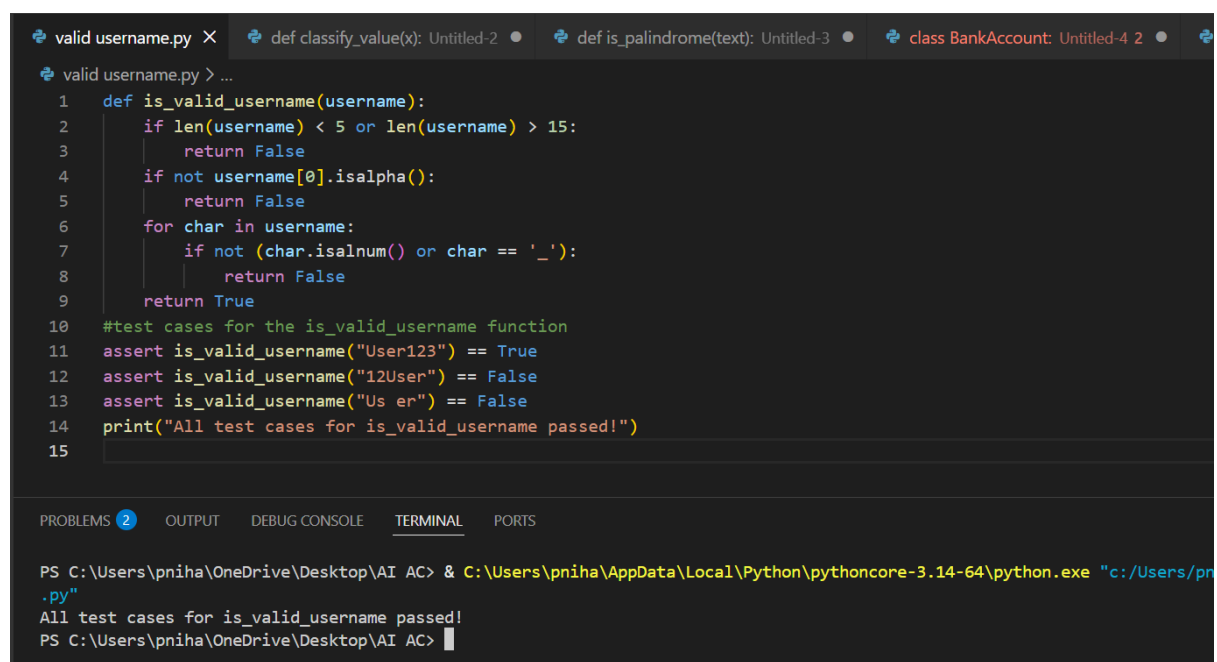
```
assert is_valid_username("User123") == True
```

```
assert is_valid_username("12User") == False
```

```
assert is_valid_username("Us er") == False
```

Expected Output #1:

- Username validation logic successfully passing all AI-generated test cases.



```
valid username.py X  def classify_value(x): Untitled-2  def is_palindrome(text): Untitled-3  class BankAccount: Untitled-4 2  ...  
valid username.py > ...  
1  def is_valid_username(username):  
2      if len(username) < 5 or len(username) > 15:  
3          return False  
4      if not username[0].isalpha():  
5          return False  
6      for char in username:  
7          if not (char.isalnum() or char == '_'):  
8              return False  
9      return True  
10 #test cases for the is_valid_username function  
11 assert is_valid_username("User123") == True  
12 assert is_valid_username("12User") == False  
13 assert is_valid_username("Us er") == False  
14 print("All test cases for is_valid_username passed!")  
15  
PROBLEMS 2  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  
PS C:\Users\pniha\OneDrive\Desktop\AI AC> & C:\Users\pniha\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/pniha/AppData/Local/Python/pythoncore-3.14-64/python.exe" "c:/Users/pniha/Desktop/AI AC/valid_username.py"  
All test cases for is_valid_username passed!  
PS C:\Users\pniha\OneDrive\Desktop\AI AC>
```

Task Description #2 (Even–Odd & Type Classification – Apply AI for Robust Input Handling)

- Task: Use AI to generate at least 3 assert test cases for a function `classify_value(x)` and implement it

using conditional logic and loops. • Requirements:

- o If input is an integer, classify as "Even" or "Odd".
- o If input is 0, return "Zero".
- o If input is non-numeric, return "Invalid Input".

Example Assert Test Cases:

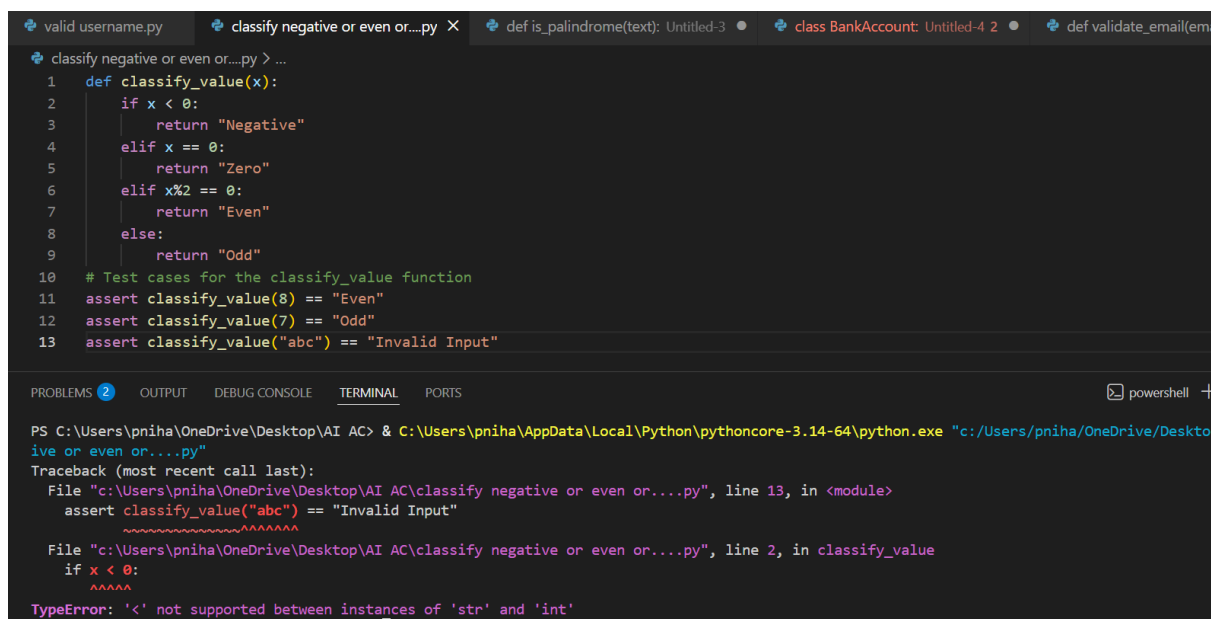
```
assert classify_value(8) == "Even"
```

```
assert classify_value(7) == "Odd"
```

```
assert classify_value("abc") == "Invalid Input"
```

Expected Output #2:

- Function correctly classifying values and passing all test cases



```
valid username.py  classify negative or even or....py X  def is_palindrome(text):  Untitled-3  class BankAccount:  Untitled-4 2  def validate_email(email):  ...

classify negative or even or....py > ...
1  def classify_value(x):
2      if x < 0:
3          return "Negative"
4      elif x == 0:
5          return "Zero"
6      elif x%2 == 0:
7          return "Even"
8      else:
9          return "Odd"
10 # Test cases for the classify_value function
11 assert classify_value(8) == "Even"
12 assert classify_value(7) == "Odd"
13 assert classify_value("abc") == "Invalid Input"

PROBLEMS 2  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS  powershell +
PS C:\Users\pniha\OneDrive\Desktop\AI AC> & C:\Users\pniha\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/pniha/OneDrive/Desktop/ive or even or....py"
Traceback (most recent call last):
  File "c:\Users\pniha\OneDrive\Desktop\AI AC\classify negative or even or....py", line 13, in <module>
    assert classify_value("abc") == "Invalid Input"
    ~~~~~^~~~~~
  File "c:\Users\pniha\OneDrive\Desktop\AI AC\classify negative or even or....py", line 2, in classify_value
    if x < 0:
    ~~~~^
TypeError: '<' not supported between instances of 'str' and 'int'
```

Task Description #3 (Palindrome Checker – Apply AI for String Normalization)

- Task: Use AI to generate at least 3 assert test cases for a function `is_palindrome(text)` and implement the function.
- Requirements:
 - o Ignore case, spaces, and punctuation.
 - o Handle edge cases such as empty strings and single

characters.

Example Assert Test Cases:

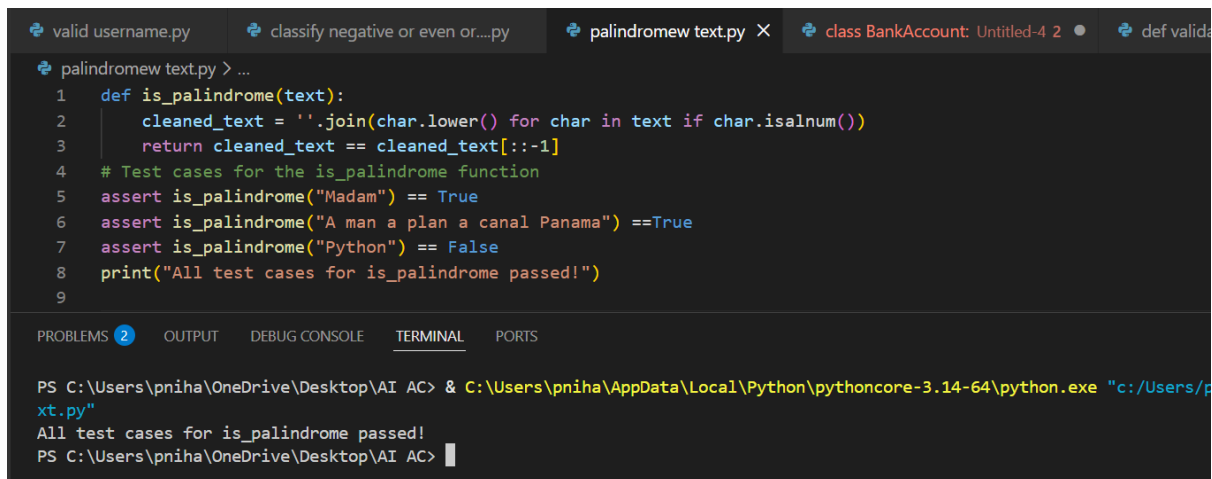
```
assert is_palindrome("Madam") == True
```

```
assert is_palindrome("A man a plan a canal Panama") == True
```

```
assert is_palindrome("Python") == False
```

Expected Output #3:

- Function correctly identifying palindromes and passing all AI-generated tests



```
valid username.py  classify negative or even or...py  palindromew text.py X  class BankAccount: Untitled-4 2  def valida

palindromew text.py > ...
1  def is_palindrome(text):
2      cleaned_text = ''.join(char.lower() for char in text if char.isalnum())
3      return cleaned_text == cleaned_text[::-1]
4  # Test cases for the is_palindrome function
5  assert is_palindrome("Madam") == True
6  assert is_palindrome("A man a plan a canal Panama") == True
7  assert is_palindrome("Python") == False
8  print("All test cases for is_palindrome passed!")
9

PROBLEMS 2  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\pniha\OneDrive\Desktop\AI AC> & C:\Users\pniha\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/pniha/Desktop/AI AC/palindromew text.py"
All test cases for is_palindrome passed!
PS C:\Users\pniha\OneDrive\Desktop\AI AC>
```

Task Description #4 (BankAccount Class – Apply AI for Object-Oriented Test-Driven Development)

- Task: Ask AI to generate at least 3 assert-based test cases for a BankAccount class and then implement the class.

- Methods:

- o deposit(amount)

- o withdraw(amount)

- o get_balance()

Example Assert Test Cases:

```
acc = BankAccount(1000)
```

```
acc.deposit(500)
```

```
assert acc.get_balance() == 1500
```

```
acc.withdraw(300)
```

```
assert acc.get_balance() == 1200
```

Expected Output #4:

- Fully functional class that passes all AI-generated assertions.Task Description #5 (Email ID Validation – Apply AI for Data Validation)

```

1 class BankAccount:
2     def __init__(self, account_number, balance=0):
3         self.account_number = account_number
4         self.balance = balance
5
6     def deposit(self, amount):
7         if amount > 0:
8             self.balance += amount
9             return True
10        return False
11
12    def withdraw(self, amount):
13        if 0 < amount <= self.balance:
14            self.balance -= amount
15            return True
16        return False
17
18    def get_balance(self):
19        return self.balance
20
21    # Test cases for the BankAccount class
22    acc = BankAccount(1000)
23    acc.deposit(500)
24    assert acc.get_balance() == 1500
25    acc.withdraw(300)
26    assert acc.get_balance() == 1200
27    print("All test cases for BankAccount passed!")

```

```

Traceback (most recent call last):
  File "c:\Users\pniha\OneDrive\Desktop\AI AC\bankaccount withdraw.py", line 22, in <module>
    assert acc.get_balance() == 1500
           ^^^^^^^^^^^^^^^^^^^^^^^^^
AssertionError
PS C:\Users\pniha\OneDrive\Desktop\AI AC>

```

Task Descripton #5 (Email ID Validaton – Apply AI for Data Validaton)

- Task: Use AI to generate at least 3 assert test cases for a function validate_email(email) and

implement the function.

- Requirements:
 - o Must contain @ and .
 - o Must not start or end with special characters.
 - o Should handle invalid formats gracefully.

Example Assert Test Cases:

```

assert validate_email("user@example.com") == True
assert validate_email("userexample.com") == False
assert validate_email("@gmail.com") == False

```

Expected Output #5

Email validaton functon passing all AI-generated test cases and handling edge cases correctly.

valid username.py classify negative or even or....py palindromew text.py bankaccount withdraw.py VALIDATE EMAIL.py X

VALIDATE EMAIL.py > ...

```
1 def validate_email(email):
2     if '@' not in email or '.' not in email:
3         return False
4     at_index = email.index('@')
5     dot_index = email.rindex('.')
6     if at_index < 1 or dot_index < at_index + 2 or dot_index >= len(email) - 1:
7         return False
8     return True
9 # Test cases for the validate_email function
10 assert validate_email("user@example.com") == True
11 assert validate_email("userexample.com") == False
12 assert validate_email("@gmail.com") == False
13 print("All test cases for validate_email passed!")
14
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\pniha\OneDrive\Desktop\AI AC> & C:\Users\pniha\AppData\Local\Python\pythoncore-3.14-64\python.exe "c:/Users/pniha/OneDrive/
All test cases for validate_email passed!
PS C:\Users\pniha\OneDrive\Desktop\AI AC> |
```